

API Build & Integration Status Report

IDURAR CRM — Backend Routes vs Frontend Integration Audit

Generated 20 August 2026

1. Executive Summary

This report inventories every backend API route that currently exists, checks whether each frontend page actually calls it, and gives a recommended build order for what's missing. The short version: the backend has a solid, fully working CRUD engine for **3 entities** (Customer, Invoice, Payment) plus Auth/Profile/Settings — but most of the frontend pages built on top of it were rewritten as local mock demos and are no longer wired to that working backend. Several other pages (Leads, Calls, Performance, Reports, Communication, User Management) have **no backend at all yet** — they are 100% front-end mock data.

4	6	4	6
BACKEND ENTITIES LIVE & WORKING	BROKEN ROUTES (CRASH ON LOAD)	PAGES MOCK-ONLY DESPITE WORKING BACKEND	FEATURES WITH ZERO BACKEND

Fix before building anything new: 6 routes currently crash because they import from empty module folders — `/invoice/create` , `/invoice/read/:id` , `/invoice/update/:id` , `/invoice/pay/:id` , `/payment/read/:id` , `/payment/update/:id` . See Section 4 for details.

2. Backend Endpoint Inventory

Every route currently registered in the backend, grouped by feature area.

Auth & Core (no auth token needed to call, except logout)

METHOD	PATH	CONTROLLER	STATUS
POST	<code>/api/login</code>	<code>adminAuth.login</code>	Working
POST	<code>/api/forgetpassword</code>	<code>adminAuth.forgetPassword</code>	Working
POST	<code>/api/resetpassword</code>	<code>adminAuth.resetPassword</code>	Working
POST	<code>/api/logout</code>	<code>adminAuth.logout</code>	Working
—	<code>/api/register</code>	<i>does not exist</i>	Not built — also unused by frontend
—	<code>/api/verify/:userId/:emailToken</code>	<i>does not exist</i>	Not built — also unused by frontend

Admin / Profile (auth required)

METHOD	PATH	CONTROLLER	STATUS
GET	<code>/api/admin/read/:id</code>	<code>adminController.read</code>	Working
PATCH	<code>/api/admin/password-update/:id</code>	<code>adminController.updatePassword</code>	Working
PATCH	<code>/api/admin/profile/password</code>	<code>adminController.updateProfilePassword</code>	Working
PATCH	<code>/api/admin/profile/update</code>	<code>adminController.updateProfile</code> (photo upload)	Working
—	<i>list all users</i>	<i>not exposed</i>	Not built
—	<i>create user</i>	<i>not exposed</i>	Not built
—	<i>delete / deactivate user</i>	<i>not exposed</i>	Not built

Settings (auth required) — 12 routes, fully functional

METHOD	PATH	STATUS
POST	<code>/api/setting/create</code>	Working
GET	<code>/api/setting/read/:id</code>	Working
PATCH	<code>/api/setting/update/:id</code>	Working

GET	/api/setting/search · /list · /listAll · /filter	Working
GET	/api/setting/readBySettingKey/:settingKey	Working
GET	/api/setting/listBySettingKey	Working
PATCH	/api/setting/updateBySettingKey/:settingKey?	Working
PATCH	/api/setting/upload/:settingKey?	Working
PATCH	/api/setting/updateManySetting	Working
DELETE	/api/setting/delete/:id	Commented out, not registered

CRUD-Factory Entities: Customer (client), Invoice, Payment — 9 routes each

Each entity gets: [create](#) , [read/:id](#) , [update/:id](#) , [delete/:id](#) , [search](#) , [list](#) , [listAll](#) , [filter](#) , [summary](#) . Invoice and Payment additionally get [mail](#) .

ENTITY	ROUTES	BACKEND STATUS
client (Customer)	9 routes	All working
invoice	9 routes + mail	All working (custom create/read/update/delete/list/summary)
payment	9 routes + mail	All working (custom create/update/delete/summary)

Download / File

METHOD	PATH	STATUS
GET	/download/:directory/:file	Working
GET	/public/:subPath/:directory/:file	Working

3. Frontend Integration Status

For every page: does it call the backend, and does that backend endpoint actually exist?

PAGE / FEATURE	BACKEND EXISTS?	FRONTEND STATUS	NOTE
Login / Forgot / Reset / Logout	Yes	Integrated	Fully working, verified
Profile	Yes	Integrated	Photo upload + password update working
Customer	Yes (9 routes)	Mock only	Rewritten with hardcoded seed data; real backend unused
Invoice (list/view)	Yes (9 routes)	Mock only	Hardcoded seed data; real backend unused
Invoice Create / Read / Update / Record-Payment	Yes	Broken	Imports from empty <code>modules/InvoiceModule/*</code> — crashes if opened
Payment (list/view)	Yes (9 routes)	Mock only	Hardcoded seed data; real backend unused
Payment Read / Update	Yes	Broken	Imports from empty <code>modules/PaymentModule/*</code> — crashes if opened
Settings	Yes (12 routes)	Mock only	"Save" just flashes a message locally, nothing persisted
Dashboard	No	No backend	Needs new aggregation endpoints across all entities
Calls	No	No backend	Needs a new Call model + CRUD
Leads	No	No backend	Needs a new Lead model + CRUD
Performance	No	No backend	Needs Calls/Leads data first (depends on them)
Reports	No	No backend	Aggregates everything — needs all other entities first
Communication	No	No backend	Needs a new Message/Log model
User Management	Partial	Mock only	Only read/password/profile exist server-side — no list/create/delete-user routes at all

4. Fix First — Currently Broken (before any new API work)

These 6 routes are wired into the router but import from completely empty folders. Opening them in the app will fail to load / crash:

ROUTE	BROKEN IMPORT
/invoice/create	@/modules/InvoiceModule/CreateInvoiceModule (folder is empty)
/invoice/read/:id	@/modules/InvoiceModule/ReadInvoiceModule (folder is empty)
/invoice/update/:id	@/modules/InvoiceModule/UpdateInvoiceModule (folder is empty)
/invoice/pay/:id	@/modules/InvoiceModule/RecordPaymentModule (folder is empty)
/payment/read/:id	@/modules/PaymentModule/ReadPaymentModule (folder is empty)
/payment/update/:id	@/modules/PaymentModule/UpdatePaymentModule (folder is empty)

Fix options: either rebuild these modules to call the (already working) invoice/payment backend, or remove the dead routes until they're rebuilt.

5. Recommended Build Priority Order

Ranked by effort vs value — reconnect what already works before building anything new, then build new backend in dependency order.

Phase 0 — Fix broken routes (do this first, ~1 day)

- Fix or remove the 6 broken Invoice/Payment sub-routes listed in Section 4.

Why first: these currently crash. Nothing else matters if the app breaks on these clicks.

Phase 1 — Reconnect existing working APIs (no new backend needed)

- 1. **Customer** — wire the mock page back to the real `client` API
- 2. **Invoice** — wire back to the real `invoice` API (create/read/update/pay/mail)
- 3. **Payment** — wire back to the real `payment` API
- 4. **Settings** — wire "Save" buttons to the real `setting` API

Why: backend is already 100% built and tested for these four — this is pure frontend work, fastest possible win, and it's core business data (customers/invoices/payments) that should not be living only in mock state.

Phase 2 — New backend: foundational entities

- 5. **User Management** — add list/create/update-role/deactivate routes for Admin (currently only read + password exist)
- 6. **Leads** — new Lead model + full CRUD
- 7. **Calls** — new Call model + CRUD (call logs, status, duration)

Why: these are core CRM data entities other features depend on, and User Management is actively being used with mock passwords/roles right now — highest risk of confusion if left mock.

Phase 3 — Depends on Phase 2 data

- 8. **Performance** — scoring/analytics built from real Calls + Leads data
- 9. **Communication** — message/email/WhatsApp log model, linked to Leads/Customers

Why: these features are only meaningful once there's real Calls/Leads data to summarize or reference.

Phase 4 — Aggregation layer (build last)

- 10. **Reports** — cross-entity aggregation endpoints (calls, leads, invoices, payments, team performance)
- 11. **Dashboard** — summary KPIs, reusing the same aggregation endpoints as Reports

Why: both pages roll up data from everything above — building them first would mean rebuilding them again once the underlying entities land.

Cleanup item (low priority, no rush): the `register` / `verify` auth actions exist in frontend Redux but call backend routes that were never built, and no UI currently triggers them — safe to delete as dead code, or build out only if self-service signup is actually planned.